UNIVERSIDAD
DE LA RIOJA

Facultad de Ciencia y Tecnología

TRABAJO FIN DE GRADO

Grado en Ingeniería Informática

Sistema de Validación en Facturación de Servicios Cloud

Cloud Services Billing Validation System

Realizado por:

Pablo Galilea Valverde

Tutelado por:

Jesús María Aransay Azofra

Logroño, Curso 2025-2026

Dedicado a...

A mis padres y mi hermano por su apoyo incondicional.

Resumen

Palabras clave: automatización, testing, facturación, hosting, validación.

En este proyecto implementaremos una solución para validar la corrección de un sistema de facturación de servicios en la nube, que funcione de manera automatizada. Se desarrollará una API REST en Node.js que valide de manera cruzada los cargos que se hacen a través de APIs internas de la empresa, que garantice una facturación estable, fiable y basada estrictamente en el consumo de los clientes. El proyecto debe ser capaz de usar flujos de trabajo y servicios en la nube, y abordar temas como la implementación, operación y monitoreo de aplicaciones REST de back-end.

Abstract

Keywords: automation, testing, billing, hosting, validation.

This project implements an automated quality assurance solution to validate Cloud billing accuracy. A Node.js REST API application regularly cross-checks product charges via internal APIs, ensuring stable and reliable consumption-based invoicing. The project provided hands-on experience in QA workflows and Cloud Services, as well as in the topic of implementing, operating and monitoring the back-end REST applications.

Agradecimientos

En primer lugar, me gustaría expresar mi más sincero agradecimiento a mis tutores, Jesús María Aransay Azofra y José Luis Fernández Villar, por su dedicación, tiempo y valioso apoyo durante la realización de este trabajo.

Quiero extender mi agradecimiento también a Artem Levik, quien durante mis prácticas ejerció como tutor y me enseñó tanto, así como a todo el equipo de Cloud Billing & ERP de IONOS Cloud, por su profesionalidad y apoyo. Además de mis compañeros dentro de Arsys, en especial a Luisa por darme consejos de LaTeX para realizar el TFG.

Agradezco profundamente a mi familia por ser mi pilar fundamental. A mi madre, Ramoni, por ser ese apoyo incondicional en cada paso; a mi padre, Neftalí, por transmitirme su pasión por las ciencias y su facilidad para entenderlas; y a mi hermano Gael —que ya no es tan pequeño— por ser la persona que me inspira a ser mejor cada día.

Un especial reconocimiento a mi abuelo Martín, quien nunca dudó de que su nieto lograría este objetivo. Él, sin estudios formales, me enseñó más matemáticas que muchos maestros. Y, por supuesto, a mi abuela, cuyo amor y orgullo —presumiendo por Torremolinos de su nieto que ya casi era ingeniero— me dieron la energía necesaria para seguir adelante.

También quiero expresar mi gratitud a mi segunda familia: mis amigos de la universidad. Este último paso, este proyecto, es en gran parte gracias a ellos, por todas las tardes de sábados y domingos que pasamos juntos en la biblioteca. Gracias a Izai, Irune, Diego, Maider, Rubén y Urretxo por haber sido lo mejor que me llevo de mi experiencia universitaria. Una mención especial para Jacobo, por todos los días que hemos acabado mano a mano, porque sé que, aunque todo falle, tú siempre estarás ahí.

Por último, y más importante, quiero darle las gracias a Cristina. Una gran parte de este proyecto es gracias a ti. Has sido un apoyo incondicional todos estos meses, en los que a veces no me veía capaz de sacarlo adelante. Con una sonrisa, por pequeña que fuese, me animabas a seguir. Has sido la forma más bonita en que la vida me ha mostrado la felicidad. Gracias a ti no me he rendido. Te quiero.

Con todo mi cariño,
Pablo

Índice general

Resumen III

Abstract III

Agradecimientos IV

1. Introducción VIII
- 1.1. Motivación . . . VIII
- 1.2. Antecedentes . . . VIII
- 1.3. Objetivos . . . IX
  - 1.3.1. Objetivo General . . . IX
  - 1.3.2. Objetivos Específicos . . . IX
- 1.4. Metodología del Proyecto . . . IX
- 1.5. ¿Qué es el DISEÑADOR DE CENTROS DE DATOS (DCD) de IONOS Cloud? . . . X
  - 1.5.1. Definición y Propósito . . . X
  - 1.5.2. Funciones y Herramientas del DCD . . . XI
- 1.6. ¿Qué es una API REST y una API Canary? . . . XIII
  - 1.6.1. ¿Qué es una API REST? . . . XIII
  - 1.6.2. Principios REST . . . XIII
  - 1.6.3. Concepto y Origen de API Canary . . . XIII
  - 1.6.4. Características Principales . . . XIV
  - 1.6.5. Ventajas en Sistemas de Facturación . . . XIV
  - 1.6.6. Flujo de Trabajo Canary . . . XIV
- 1.7. Tecnologías y Herramientas Utilizadas . . . XVI

2. Análisis XIX
- 2.1. Definición y alcance del proyecto . . . XIX
- 2.2. Requisitos . . . XXI
- 2.3. Estudio de viabilidad . . . XXIII
  - 2.3.1. Requisitos mínimos . . . XXIII
  - 2.3.2. Viabilidad de constancia de recursos a analizar . . . XXIII

3. Planificación XXIV
- 3.1. Descomposición del Proyecto (EDT) . . . XXIV
  - 3.1.1. Explicación del EDT . . . XXV
- 3.2. Estimación de tiempos . . . XXVII
- 3.3. Cronograma e hitos . . . XXVIII
- 3.4. Plan de contingencias y riesgos . . . XXX

4. Iteraciones **XXXI**
4.1. Primera Iteración XXXI
4.1.1. Planificación XXXI
4.1.2. Desarrollo y despliegue XXXIII
4.1.3. Integración del código XXXVIII
4.1.4. Implementación de función de Detección de items continuables . XLIII

1. Lista de SKUs **XLIV**

2. Crear cuenta y obtener token **XLVII**
2.1. Creación de la Cuenta XLVII
2.2. Obtener token XLVIII

Índice de figuras

1.1. Interfaz inicial DCD IONOS Cloud XI
1.2. Interfaz en Centro de Datos del DCD IONOS Claud XI
1.3. Diagrama de flujo del proceso de despliegue Canary para APIs XV

3.1. EDT XXIV

4.1. Prueba Test XXXVII
4.2. Prueba Resumen Test XXXVIII
4.3. Prueba Cods. Test XXXVIII

Índice de cuadros

1.1. Características de una API Canary XIV
1.2. Stack tecnológico del proyecto XVII

2.1. Métricas de calidad del proyecto XXI
2.2. Requisitos no funcionales XXI
2.3. Requisitos funcionales XXII

3.1. Plan de dedicación de tiempos XXVII
3.2. Cronograma de tareas del TFG XXVIII
3.3. Cronograma de tareas del TFG XXVIII
3.4. Hitos TFG XXIX
3.5. Plan de contingencias y riesgos XXX
3.6. Plan de contingencias y riesgos XXXI

Capítulo 1

Introducción

Este capítulo presenta la motivación, el contexto, objetivos y fundamentos del trabajo desarrollado. Se expone la problemática abordada, se definen los objetivos principales y secundarios, y se introduce la tecnología base sobre la cual se asienta el proyecto: el sistema DCD de IONOS Cloud y las APIs Canary.

1.1. Motivación

La empresa promotora de este proyecto es Arsys, perteneciente al grupo IONOS, quien identificó la necesidad tras la creación y posterior ampliación del *DCD*[1], Al tratarse de un componente fundamental para la compañía, se hizo necesario establecer mecanismos de control que garantizaran su correcto funcionamiento y permitieran su verificación continua.

El subsistema de facturación asociado al *DCD* —encargado de gestionar los cobros derivados del uso y alquiler de las máquinas físicas que soportan las máquinas virtuales— constituye un foco crítico de posibles errores. Esta vulnerabilidad se debe, principalmente, a la volatilidad de los tipos de cambio de divisas y a la irregularidad en la duración de los meses, factores que pueden introducir inconsistencias en los cargos calculados.

1.2. Antecedentes

Como antecedentes se debe partir desde dos puntos de vista: tanto mis antecedentes como los de la empresa.

- **Mis antecendes:** Durante mi periodo de prácticas en la empresa, tuve la oportunidad de familiarizarme con el DCD de IONOS y de trabajar con las APIs asociadas a este entorno. Esta experiencia incluyó el desarrollo de pequeños proyectos que me permitieron conocer en profundidad la tecnología subyacente, siendo el más relevante la implementación de un detector de identificadores de recursos de otro servicio de IONOS Cloud a partir de una lista estática.

- **Antecedentes de la empresa:** Al ser una empresa grande, no es el primer servicio que automatizan, por lo que cuentan con librerías internas y herramientas tanto para automatizar como para hacer que ciertas APIs se ejecuten periódicamente. Por ello, la parte correspondiente a la creación de automatizaciones puede

[1] Data Center Designer: plataforma gráfica que permite a los clientes de IONOS diseñar y configurar sus propios centros de datos de forma visual.

apoyarse en las herramientas de la empresa, lo cual facilitará el trabajo en gran medida.

1.3. Objetivos

1.3.1. Objetivo General

Se consideró necesario desarrollar e implementar un sistema de gestión de automatización de tests para los sistemas de negocio de IONOS, específicamente enfocado en la validación de procesos de facturación cloud. Siendo el principal objetivo del proyecto comprobar de forma automática el correcto funcionamiento de la facturación del DCD.

1.3.2. Objetivos Específicos

1. **Analizar** el estado actual de los sistemas de facturación y validación en IONOS Cloud.
2. **Diseñar** una arquitectura de automatización de tests que permita la validación cruzada entre sistemas.
3. **Implementar** una API REST para la ejecución y gestión de tests automatizados.
4. **Desarrollar** un sistema de detección temprana de inconsistencias mediante técnicas de validación periódica, esto es, comprobaciones automatizadas ejecutadas en intervalos regulares (diarias, semanales o mensuales) para contrastar los cargos facturados contra el consumo real..
5. **Evaluar** la efectividad del sistema propuesto en términos de precisión, rendimiento y reducción de errores.
6. **Documentar** el proceso de desarrollo y los resultados obtenidos para su posible replicación o mejora.

1.4. Metodología del Proyecto

La metodología que se ha optado por usar, ya que se ha considerado la más adecuada, es una implementación iterativa incremental. La razón para tomar esta decisión es el hecho de que este proyecto es ideal para esta metodología. Esta se basa en la partición del desarrollo en pequeñas fases, permitiendo comprobar en cada iteración que una parte del proyecto ya es funcional y facilitando la realización de ajustes antes de continuar con funcionalidades nuevas.

Para cada iteración se seguira un proceso estandar:

- **Definición de objetivos:** En esta parte se definen los módulos del proyecto que se quieren completar, las funciones que se desea que funcionen y en concreto las tareas a realizar.

- **Análisis:** Se realizán los análisis técnicos necesarios para completar las tareas planeadas en la Definición de objetivos.
- **Desarollo:** Se llevan a cabo las implementaciones necesarias para lograr los objetivos.
- **Comprobaciones:** Se hacen test, que pueden ser tanto manuales como automáticos, para comprobar que funcionan correctamente las funciones deseadas, en un contexto ideal se comprueban las funciones de las iteraciones anteriores, para asegurar no haber dañado ninguna funcionalidad anterior.
- **Valoración:** Tras cada iteración, se evalua si se han cumplido los objetivos y en caso negativo las causas.

La metodología adoptada ofrece flexibilidad para adaptarse a cambios en los requisitos, entregas parciales que generan prototipos funcionales para ajustes iterativos, reducción de riesgos mediante identificación temprana de problemas en ciclos cortos, mejora continua de la calidad a través de revisiones periódicas, y control detallado del progreso para una gestión eficiente de recursos. Esto permite un desarrollo incremental y dinámico donde las funcionalidades se implementan progresivamente.

1.5. ¿Qué es el DISEÑADOR DE CENTROS DE DATOS (DCD) de IONOS Cloud?

1.5.1. Definición y Propósito

El **Data Center Designer (DCD)** de IONOS Cloud es una plataforma gráfica que permite a los clientes de IONOS diseñar y configurar sus propios centros de datos de forma visual. Permite a los clientes:

- **Provisionamiento de recursos**: Servidores virtuales, almacenamiento, redes.
- **Gestión del ciclo de vida**: Creación, modificación y eliminación de recursos.
- **Monitorización**: Supervisión del estado y rendimiento de la infraestructura.
- **Facturación**: Cálculo y generación de cargos basados en consumo.

Es muy intuitivo y tiene una interfaz bien desarrollada como se puede observar en la imagen.

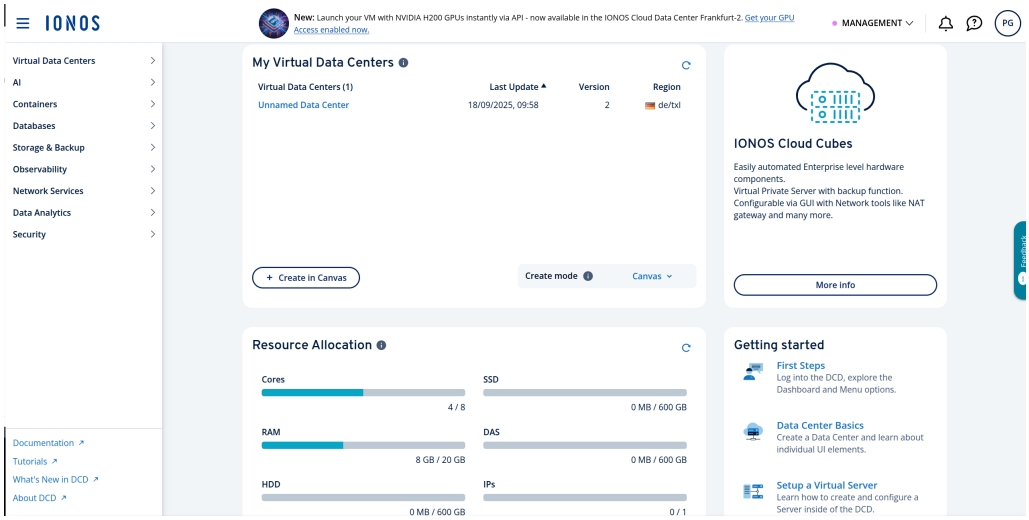


Figura 1.1: Interfaz inicial DCD IONOS Cloud

1.5.2. Funciones y Herramientas del DCD

La arquitectura del DCD se compone de múltiples microservicios que interactúan a través de la capa gráfica una vez dentro del diseñador, como se muestra en la imagen.

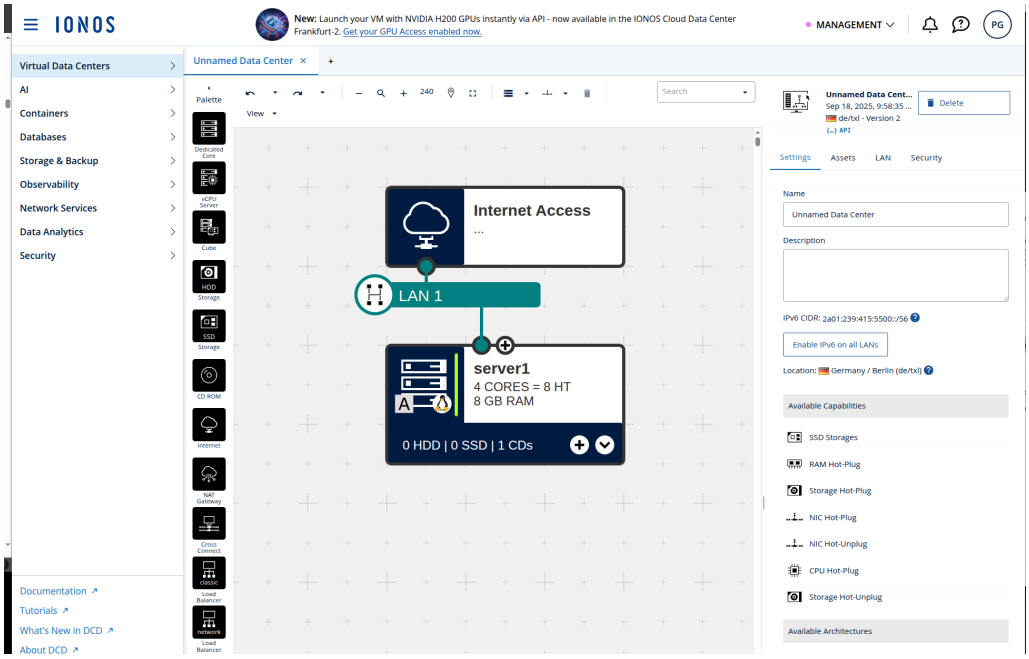


Figura 1.2: Interfaz en Centro de Datos del DCD IONOS Claud

Herramientas del DCD

El DCD tiene muchas herramientas y es importante conocerlas para este proyecto, porque para controlar la facturación hay que saber los tipos de elementos que forman el DCD.

1. **Diseñador Grafico:** Interfaz principal en la que se pueden añadir los recursos principales que se desgloban en el Anexo 1.

2. **Inteligencia Artificial:** Tiene una herramienta de apoyo de inteligencia artificial de varios modelos; se mide su uso por medio de *tokens*[2].

3. **Contenedores:** es la categoría que agrupa las herramientas para la modernización de aplicaciones (cloud-native). Ofrece dos servicios esenciales y sinérgicos:

   - **Container Registry**: El "DÓNDE"guardas tus componentes de software empaquetados (imágenes).
   - **Managed Kubernetes:** El ÇÓMO"los pones en producción de forma automática, escalable y resistente.

4. **Bases de datos:** Conjunto de datos organizado de tal modo que permita obtener con rapidez diversos tipos de información. Hay tanto relacionales como no relacionales, para más información consultar el anexo 1.

5. **Almacenamiento y Copias de Seguridad :** En el *DCD*, Storage y Backup son pilares complementarios de la estrategia de datos, siendo el Almacenamiento donde "viven"mis datos ahora mismo y las copias de seguridad la manera de recuperar dicha información. Ofreciendo cinco servicios:

   - **IONOS Object Storage:** Almacén masivo para archivos y objetos.
   - **Network File Storage:** Sistema de archivos compartido en red.
   - **Backup Unit Manager:** Automatizar y gestionar copias de seguridad.
   - **Images y Snapshots:** Congelar el estado para revertir o clonar.
   - **FTP Image Upload:** Importar plantillas personalizadas de sistemas.

6. **Servicios de red:** Define cómo se conectan y comunican todos los recursos, es una de las partes más importantes, desglosandose en varias partes:

   - **Administración de direcciones IP:** gestión centralizada de direcciones IP.
   - **Puerta de Enlace VPN:** Servicio que crea un túnel cifrado y seguro entre tu infraestructura en IONOS Cloud y otro sitio.

[2]Un *token* es una cadena alfanumérica única y secreta generada por un servidor para autenticar y autorizar a un cliente (aplicación, usuario o dispositivo) al realizar solicitudes

- **Red de Distribución de Contenidos:** Una red global de servidores que almacenan en caché copias de tu contenido estático (imágenes, CSS>, JS>, videos) cerca de tus usuarios finales.
- **Cloud DNS:** El servicio de resolución de nombres de dominio gestionado por IONOS. Traduce nombres de dominio (www.tuempresa.com) a direcciones IP (185.100.100.10).

7. **Seguridad** No es un producto único, sino un marco de trabajo integrado que aplica el principio de "defensa en profundidad"basandose en:

   - **Manager de llaves SSH:** protege el "QUIÉN"; Autenticación fuerte para administradores y automatización.
   - **Grupos de Seguridad de Red:** Protege el "QUÉ y DÓNDE"; Control estricto del flujo de tráfico entre los recursos.
   - **Gestor de certificados:** Protege el ÇÓMO"; Confianza y privacidad para las comunicaciones con los usuarios finales.

1.6. ¿Qué es una API REST y una API Canary?

1.6.1. ¿Qué es una API REST?

Es un intermediario estandarizado que permite que diferentes aplicaciones (web, móvil, sistemas legacy) se comuniquen entre sí usando el protocolo HTTP, siguiendo principios específicos que la hacen escalable, simple y stateless.

1.6.2. Principios REST

- **Arquitectura Cliente-Servidor:** Hay una clara separación entre el cliente(solicita) y el servidor (responde)
- **Sin Estado:** Cada petición contiene toda la información necesaria.
- **Cacheable:** Significa que las respuestas de una API REST deben estar diseñadas para poder ser almacenadas temporalmente (cacheadas) por clientes.
- **Interfaz Uniforme:** Todo es un recurso con URI única.
- **Sistema en Capas:** El cliente no sabe si está conectado al servidor final o a un intermediario.

1.6.3. Concepto y Origen de API Canary

El término **Çanary"** proviene de la práctica minera de usar canarios para detectar gases peligrosos. En desarrollo de software, una **API Canary** se refiere a:

Una implementación de API que se despliega gradualmente a un subconjunto de usuarios o sistemas para detectar posibles problemas antes de que el usuario final lo detecte. El método de funcionamiento es una ejecución periodica con una alarma si falla, la frecuencia de dicha ejecución sera dependiente de lo critico que sea el módulo que controla.

1.6.4. Características Principales

Característica	Descripción
Despliegue Gradual	Se implementa primero a un pequeño porcentaje del tráfico.
Monitorización Intensiva	Se supervisa exhaustivamente en busca de errores.
Rollback Automático	Si se detectan problemas, se revierte automáticamente.
A/B *Testing*[3]	Permite comparar versiones nuevas vs. antiguas.

Tabla 1.1: Características de una API Canary

1.6.5. Ventajas en Sistemas de Facturación

La implementación de APIs Canary en sistemas de facturación ofrece:

- **Detección temprana**: Problemas se identifican rápidamente, el tiempo que tarde en detectarse sera dependiente de la frecuencia de su ejecución.
- **Reducción de riesgos**: Errores de facturación afectan solo a un subconjunto de clientes. Ya que al detectarse de forma temprana y le afecta a menos clientes.
- **Validación en producción**: Pruebas con datos reales sin comprometer todo el sistema. Ya que se crea un cliente real completo para usar en la API.
- **Mejora continua**: Permite iteraciones rápidas y seguras. Ya que es muy fácil de separar en módulos.

1.6.6. Flujo de Trabajo Canary

Descripción del flujo:

1. **Inicio: Nueva API Canary** - Desarrollo de una nueva API de validación.
2. **Inicio: Nueva Versión API Canary** - Desarrollo de una nueva versión de la API de validación. En la que se inicia la nueva iteración.

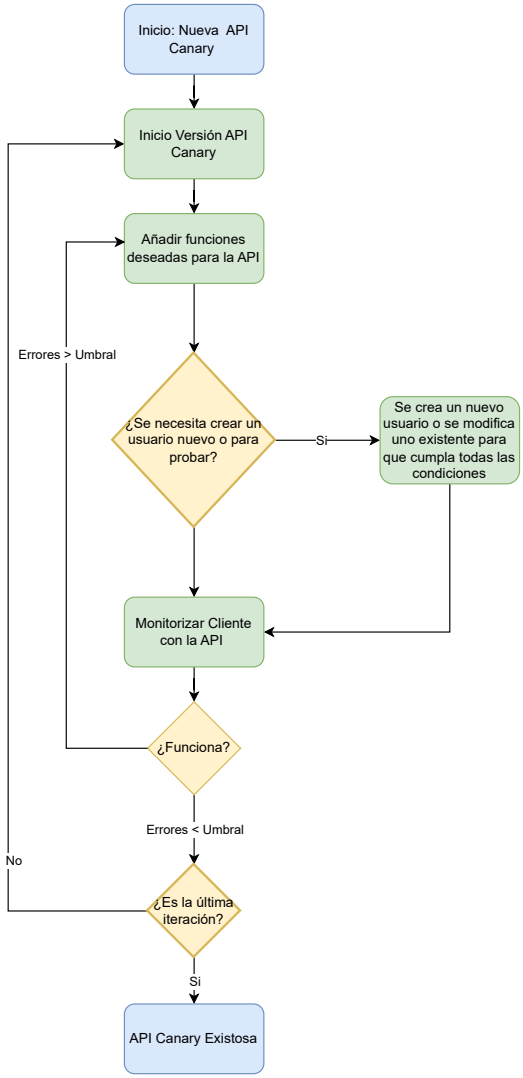


Figura 1.3: Diagrama de flujo del proceso de despliegue Canary para APIs

3. **Inicio: Añadir funciones deseadas** - Se implementan las nuevas funciones deseadas.

4. **Comprobación de completitud del usuario de muestra** - Se comprueba que el usuario que se usa para comprobar el correcto funcionamiento de la API tiene todos los recursos necesarios que debe comprobar. En el caso contrario, se

modifica para que permita comprobar todas las funciones a comprobar, en su defecto se creara un nuevo perfil para ello.

5. **Monitorizar Métricas** - Análisis continuo de errores, latencia y otras métricas clave.

6. **Decisión basada en umbrales:**

   - **Si errores <Umbral** → Se pasa a la siguiente fase de la API.
   - **Si errores >Umbral** → Rollback automático a versión estable y se revisan las actualizaciones para volver a modificar y lograr hacer funcionales los cambios.

7. **API Canary Exitosa** - Tras terminar todas las fases, se integra y entra la fase de automatizarla para que se ejecute periodicamente.

1.7. Tecnologías y Herramientas Utilizadas

Para el desarrollo del sistema de validación de facturación cloud, se ha construido una arquitectura tecnológica robusta que combina herramientas modernas de desarrollo, infraestructura contenerizada y patrones de diseño específicos para el dominio de facturación.

Stack Tecnológico Principal

Dependencias Clave del Proyecto

Dependencias Internas:

- `@internal/core` - Framework base con utilidades de inicialización y configuración
- `@internal/dev` - Suite de herramientas para desarrollo y debugging

Dependencias Externas Críticas:

- `helmet` - Middleware de seguridad para cabeceras HTTP
- `express` - Framework web minimalista para Node.js
- `couchdb-nano` - Cliente para integración con CouchDB

Categoría	Tecnologías
Runtime	Node.js 18+ (LTS)
Lenguaje	JavaScript ES6+ con módulos
Framework Web	Express.js con middleware personalizado
Gestión de Paquetes	Yarn 3+ con workspace
Testing	Postman
Creación de usuarios de muestreo	DCD de IONOS
Base de Datos Configuración	CouchDB para configuración distribuida
Control de Versiones	GitLab para paquetes internos
Editor de Código	Visual Studio Code
Documentación	LaTeX en OverLeaf
Creación de diagramas	Draw.io
Conexión con equipo	VPN a red Alemana

Tabla 1.2: Stack tecnológico del proyecto

Arquitectura Modular Implementada

La arquitectura del sistema se organiza en tres capas principales:

1. **Capa de Infraestructura:**
   - CouchDB para gestión de configuración centralizada
2. **Capa de Servicios:**
   - `data-api`: Servicio de acceso y transformación de datos
   - `invoice-validation`: Motor de validación de facturación
   - `utilization`: Procesamiento de métricas de uso
3. **Capa de Helpers y Utilidades:**
   - Módulo `schema`: Validación y transformación de datos
   - Módulo `actions`: Operaciones comunes del negocio
   - Módulo `invoice`: Lógica específica de facturación

Flujo de Desarrollo y Calidad

- **Desarrollo Local:** Nodemon para hot-reload durante el desarrollo
- **Calidad de Código:** ESLint con configuración personalizada
- **Control de Cambios:** Hooks pre-commit para validaciones automáticas
- **Integración:** Pipelines CI/CD en GitLab para despliegue continuo

Capítulo 2

Análisis

En este capítulo se define el proyecto, creando los requisitos para la identificación de los requisitos funcionales y no funcionales. Después de obtener dichos requisitos, se limitará el proyecto mediante un estudio de viabilidad debido a la limitación de recursos, en concreto el tiempo. En caso de ser viable, se realizará y en caso negativo se limitará a una parte de dicho proyecto que proporcione un conjunto modular funcional.

2.1. Definición y alcance del proyecto

Definición del Proyecto

El proyecto **API Canary de Validación de Facturación Cloud** consiste en el desarrollo e implementación de un sistema de validación cruzada automatizada para los procesos de facturación de IONOS Cloud. Esta solución implementa una arquitectura de despliegue gradual (Canary) que permite la detección temprana de inconsistencias mediante la comparación de resultados entre sistemas independientes.

La API actúa como un *sistema de alerta temprana* que ejecuta validaciones periódicas sobre los datos de facturación generados por el Data Center Designer (DCD), comparándolos con los sistemas legacy de facturación para garantizar coherencia y precisión en los cargos a los clientes.

Alcance Técnico

- **Desarrollo de API REST:** Implementación de *endpoints*[1] para ejecución, programación y monitoreo de validaciones.
- **Sistema de Despliegue Canary:** Mecanismos para implementaciones graduales con capacidad de rollback automático. Basado en GitLab.
- **Validación Cruzada:** Comparación automática entre:
  - Datos del DCD (Data Center Designer).
  - Cálculos teóricos reales del coste.
- **Monitoreo y Alertas:** Sistema de notificaciones en tiempo real ante discrepancias detectadas.

[1] Es la URL específica y única (https://api.ejemplo.com/usuarios) que permite a una aplicación cliente interactuar con un servidor para acceder, modificar o enviar recursos específicos.

Alcance Funcional

El sistema cubre las siguientes funcionalidades principales:

1. **Validación Automatizada:**
   - Ejecución programada de validaciones (diaria, semanal, mensual).
   - Verificación de consistencia en cálculos de facturación.
   - Detección de discrepancias en precios y tarifas.
2. **Gestión de Configuración:**
   - Administración de reglas de validación.
   - Configuración de umbrales de tolerancia.
   - Gestión de frecuencias de ejecución.
3. **Reporting y Dashboard:**
   - Generación de reportes de validación.
   - Dashboard de métricas y KPIs.
   - Histórico de discrepancias detectadas.
4. **Integraciones:**
   - Conexión con sistemas DCD de IONOS.
   - Comunicación con sistemas de alertas (Slack, Email, Teams).

Límites y Exclusiones

- **No incluye:** Modificación de sistemas legacy de facturación.
- **No incluye:** Desarrollo de interfaz gráfica de usuario (GUI) completa.
- **No incluye:** Gestión de reclamaciones de clientes.
- **Se limita a:** Entornos de desarrollo, testing y producción de IONOS Cloud.
- **Se limita a:** Validación de facturación del DCD de IONOS cloud (no otros productos).

Objetivos de Calidad

Métrica	Objetivo	Justificación
Tiempo de detección de errores	<24 horas	Minimizar impacto en clientes
Precisión en validación de elementos estáticos	99.9 %	Garantizar fiabilidad de facturación
Cobertura de tests	>95 %	Calidad del código y funcionalidad

Tabla 2.1: Métricas de calidad del proyecto

Entregables del Proyecto

- **Código En este capítulo se define el proyecto, creando los requisitos para la identificación de los requisitos funcionales y no funcionales. Después de obtener dichos requisitos se limitará el proyecto teniendo, por medio de un estudio de viabilidad debido a la limitación de recursos, en concreto el tiempo. En caso de ser viable se realizara y en caso negativo se limitara a una parte de dicho proyecto que proporcione un cpnjunto modular funcional. fuente:** Repositorio Git con toda la implementación.
- **Memoria de implementación:** Documentación del proceso y de lecciones aprendidas.

2.2. Requisitos

A partir del punto anterior se desarrollarán los siguientes requisitos:

Requisitos no Funcionales	
Código	Descripción
RNF01	Fiabilidad, tener garantías del correcto funcionamiento
RNF02	Mantenibilidad, teniendo que ser fácil la actualización y fácil adición de nuevos tipos de recursos
RNF03	Seguridad, Credenciales seguras para las APIs y accesos restringidos al contrato de pruebas.
RNF04	Costos, minimización de los costos de recursos de prueba

Tabla 2.2: Requisitos no funcionales

Requisitos Funcionales	
Código	Descripción
RF01 - Gestión del Contrato de Pruebas	
RF01.01	El sistema debe tener un usuario exclusivo para las pruebas de facturación.
RF01.02	El contrato del usuario debe tener todos los recursos facturables del producto.
RF01.03	Exclusión manual de productos con costos de licencia elevados cuando sea posible
RF02 - Recursos de Consumo Constante (Continuables)	
RF02.01	Provisión automática de recursos que mantienen consumo constante
RF02.01.01	Provisión automática de Maquinas Virtuales (VMs)
RF02.01.02	Provisión automática de Almacenamiento (S3 buckets, discos)
RF02.02	Configuración única sin modificaciones posteriores
RF02.03	Consumo mensual predecible y estable, no introducir cambios bruscos
RF03 - Generación de Consumo Variable (No Continuables)	
RF03.01	Servicio Canary API para generar consumo artificial
RF03.02	Generación de cantidades similares mensualmente
RF03.03	Tolerancia configurable para variaciones esperadas
RF03.04	Programación de jobs cron para generación periódica
RF04 - Verificación de Facturación	
RF04.01	Recolección diaria de datos de consumo
RF04.02	Comparación con cálculos dinámicos generados automáticamente
RF04.03	Márgenes de tolerancia configurables por *SKU*[2]
RF05 - Sistema de Alertas y Reportes	
RF05.01	Notificaciones automáticas en desviaciones significativas
RF05.04	Histórico de verificaciones y desviaciones

Tabla 2.3: Requisitos funcionales

2.3. Estudio de viabilidad

2.3.1. Requisitos mínimos

- **Usuario con contrato:** Se necesita un contrato con capacidad de recursos mayor a un contrato estándar del DCD de IONOS, ya que se necesitan recursos extra para poder lograr usar todos los elementos a estudiar. En este caso no hay problema, ya que al ser el DCD de IONOS un servicio nuestro, se puede solicitar dicho contrato ampliado.

- **Acceso a APIs internas:** Se necesita acceso a estas APIs como por ejemplo *IONOS Cloud Billing API (3.9.0)*, siendo esta imprescindible, ya que es de donde se sacarán los datos a comparar y usar. Una vez conseguido el contrato se puede adquirir de forma natural.

- **Aceso a Base de datos de configuración:** Se necesita acceso a la base de datos de *Apache CouchDB* porque es la Base de Datos usada para las configuraciones y tiene acceso restringido. El acceso a esta página web es más complejo, pero solicitándolo con tiempo es posible.

2.3.2. Viabilidad de constancia de recursos a analizar

Hay dos tipos, los *items* continuables y los no continuables, siendo los continuables, los que son recursos que todos los meses son iguales, como por ejemplo una máquina virtual o una *IP*. Por otro lado están los recursos no continuables, siendo estos los que no tienen por qué ser todos los meses iguales, como puede ser por ejemplo el uso de *tokens* de uso de una Inteligencia Artificial.

- **items continuables :** La constancia de estos recursos es sencilla, el único problema posible es añadir todos los recursos que se quieren comprobar, ya que no hay una clara vinculación entre los items y luego su representación gráfica en la interfaz del DCD.

- **items no continuables :** La continuidad de estos items es más compleja, debido a que no hay forma de que todos los meses sea exactamente igual y que uso debe ser de manera manual o automatizada cada mes, por lo que se debe hacer un estudio más desarrollado.

Total de Horas

Capítulo 3

Planificación

Hechos ya los requisitos y sabiendo la metodología que se va a seguir, el siguiente paso es planificar de forma concreta el proyecto. En esta fase se crearán los distintos módulos del proyecto (EDT). Además, se establecerán las fechas límite para terminar los diferentes entregables y el tiempo estimado para cada apartado. Además, se incluirá un diagrama de hitos para verlo de forma más visual. Finalmente, se diseñará un plan de gestión de riesgos basándose en la metodología aprendida durante la carrera para evitar o mitigar posibles problemas.

3.1. Descomposición del Proyecto (EDT)

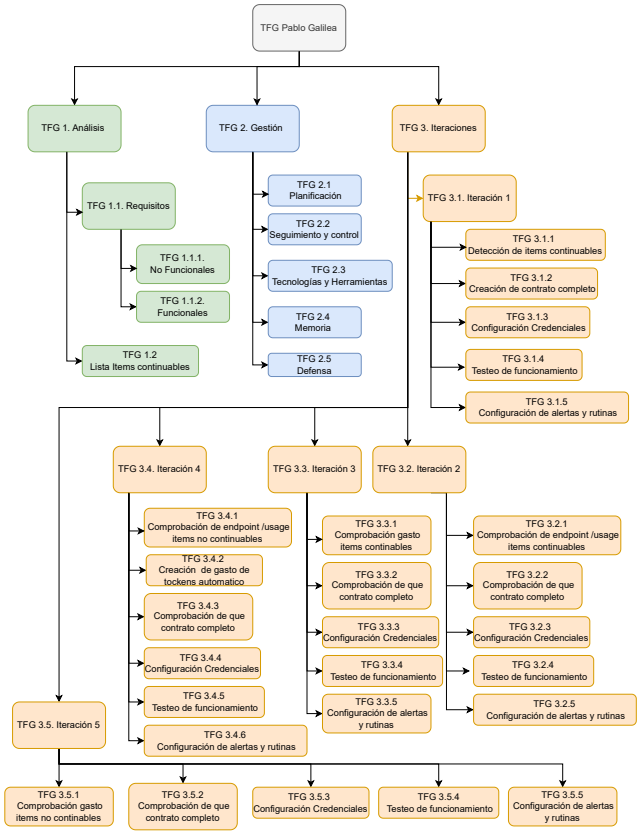


Figura 3.1: EDT

3.1.1. Explicación del EDT

En este apartado se describirán los diferentes módulos del EDT.

1. **Análisis**

   1.1 **Análisis de Requisitos :** Identificación, documentación y validación de las necesidades del proyecto para definir las funcionalidades (funcionales) y restricciones (no funcionales) de un sistema.

   1.1.1 **No Funcionales:** Lista de aquellos requisitos que aunque no añaden funcionalidad al proyecto son importantes a tener en cuenta para lograr su ejecución.

   1.1.2 **Funcionales:** Lista de aquellos requisitos necesarios para lograr que el proyecto cumpla con todas las características necesarias para que sea funcional.

   1.2 **Lista de items continuables:** Una lista con todos los SKUs de los recursos del DCD que están en funcionamiento actualmente. Se omiten los recursos que están obsoletos.

2. **Gestión**

   2.1 **Planificación:** Es el proceso de definir objetivos, tareas, recursos, costos y cronogramas para guiar la ejecución y asegurar el éxito.

   2.2 **Seguimiento y Control:** Tras la implementación y la finalización del proyecto se hace un seguimiento de su correcto funcionamiento.

   2.3 **Tecnologías y Herramientas:** Son las metodologías y enfoques que se han usado para la elaboración del proyecto.

   2.4 **Memoria** El proceso de documentación del proyecto para poder analizarlo.

   2.5 **Defensa** Argumentos y explicación sobre el proyecto para poder exponerlo a un tribunal ante posibles dudas.

3. **Iteraciones**

   3.1 **Iteración 1:** Siendo esta la primera iteración, es en la que se crea la base en la que se basarán el resto de iteraciones, por lo que se deben tomar sus decisiones teniendo en cuenta que le podría afectar al resto del proceso. En esta primera iteración se detectaran los elementos que forman un contrato.

   3.1.1 **Detección de items continuables:** La funcionalidad añadida en esta primera iteración se basa en lograr detectar que SKUs forman parte de la lista creada, como el contrato será creado por nosotros se añadirán todos los SKUs, así que todos deberán estar, todo el que falte será un falso positivo.

   3.1.2 **Creación de contrato completo:** De manera manual se crea un contrato con todos los SKUs de la lista.

3.1.3 **Configuración de Credenciales:** Esto se hace en todas las iteraciones, por lo que no se explicará en el resto, solo en este, se basa en la configuración concreta de CouchDB para el correcto funcionamiento.

3.1.4 **Testeo de funcionamiento:** Al igual que el anterior, este paso se hace en todas las iteraciones, por lo que solo se explicará en esta. En este apartado se crean las comprobaciones del correcto funcionamiento de esta y de las anteriores iteraciones.

3.1.5 **Configuración de alertas y rutinas:** Este apartado tiene la misma peculiaridad que los otros dos anteriores, por lo que se explicará solo en esta ocasión y se basa en crear las alertas que se enviarán en caso de fallo dependiendo de la información que ya se disponga, en este caso los SKUs que faltan y también crear el disparador para que se ejecute en cierto momento.

3.2 **Iteración 2:** En esta iteración ya se implementa el medir el uso de los SKUs continuables detectados en la iteración anterior y compararla con la esperada.

3.2.1 **Detección de endpoint /usage items continuables:** Para comprobar el uso de los elementos del DCD por medio de su API se puede comprobar con /usage.

3.2.2 **Comprobación de que contrato completo:** Se comprueba que no haya que añadir nada nuevo al contrato, en caso de ser necesario se añade y como será igual en el resto de iteraciones tampoco se añadirá.

3.3 **Iteración 3:** En esta iteración sabiendo el uso de los diferentes elementos se calcula su gasto económico teórico y se comprueba con el real.

3.3.1 **Comprobación gasto items continuables:** Sabiendo el gasto se calcula el precio de cada uno y se compara con el proporcionado por la API.

3.4 **Iteración 4:** En esta iteración se pasa a los items no continuables por lo que se estudia como hacer un gasto relativamente constante de estos.

3.4.1 **Comprobación de endpoint /usage items no continuables:** Se comprueba el gasto de items no continuables por medio de su SKU.

3.4.2 **Creación de gasto de tokens automático** Se estudia como lograr un gasto lo más constante posible de tokens.

3.5 **Iteración 5:**Teniendo ya el gasto se calcula su precio y se comprueba con el gasto proporcionado por la API.

3.5.1 **Comprobación gasto items no continuables** Se comprueba el gasto económico de items no continuables por medio de su SKU comparándolo con el obtenido en la API.

3.2. Estimación de tiempos

Plan de dedicación de tiempo			
Bloque	Nodo	Tiempo	Total
Análisis	Lista requisitos funcionales	5h	15h
	Análisis de requisitos Funcionales	5h	
	Análisis de requisitos no Funcionales	5h	
Cronograma de tareas Gestión	Planificación	35h	141h
	Seguimiento y control	10h	
	Tecnologías y Herramientas	6h	
	Memoria	75h	
	Defensa	15h	
Iteración 1	Detección de items continuables	16h	42h
	Creación de contrato completo	12h	
	Configuración de credenciales	6h	
	Testeo de funcionamiento	4h	
	Configuración de alertas y rutinas	4h	
Iteración 2	Comprobación de /usage items continuables	16h	21h
	Comprobación de que contrato completo	1h	
	Configuración de credenciales, alertas y rutinas	2h	
	Testeo de funcionamiento	2h	
Iteración 3	Comprobación gasto items continables	16h	23h
	Comprobación de que contrato completo	1h	
	Configuración de credenciales ,alertas y rutinas	2h	
	Testeo de funcionamiento	4h	
Iteración 4	Comprobación de /usage items no continuables	12h	37h
	Creación de gasto de tokens automático	16h	
	Comprobación de que contrato completo	1h	
	Configuración de credenciales, alertas y rutinas	4h	
	Testeo de funcionamiento	4h	
Iteración 5	Comprobación gasto items no continables	10h	21h
	Comprobación de que contrato completo	1h	
	Configuración de credenciales, alertas y rutinas	4h	
	Testeo de funcionamiento	6h	
Total de Horas		300 h	

Tabla 3.1: Plan de dedicación de tiempos

3.3. Cronograma e hitos

Cronograma de tareas													
Paquete de trabajo		**Febrero**				**Marzo**				**Abril**			
		S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12
TFG 1.1.1	Análisis de requisitos no funcionales												
TFG 1.1.2	Análisis de requisitos funcionales												
TFG 1.2	Lista Items continuables												
TFG 2.1	Planificación												
TFG 2.2	Seguimiento y control												
TFG 2.3	Tecnologías y herramientas												
TFG 2.4	Memoria												
TFG 3.1.1	Detección de items continuables												
TFG 3.1.2	Creación de contrato completo												
TFG 3.x.3	Configuración Credenciales												
TFG 3.x.4	Testeo funcionamiento												
TFG 3.x.5	Configuración de alertas y rutinas												
TFG 3.2.1	Comprobación de /usage items continuable												
TFG 3.x.2	Comprobación de que contrato completo												
TFG 3.3.1	Comprobación gasto items continables												
TFG 3.4.1	Comprobación de /usage items no continuables												
TFG 3.4.2	Creación de gasto de tockens automatico												
TFG 3.5.1	Comprobación gasto items no continables												

Tabla 3.2: Cronograma de tareas del TFG

Cronograma de tareas II													
Paquete de trabajo		**Mayo**				**Junio**				**Julio**			
		S13	S14	S15	S16	S17	S18	S19	S20	S21	S22	S23	S24
TFG 2.2	Seguimiento y control												
TFG 2.4	Memoria												
TFG 2.4	Defensa												
TFG 3.x.4	Testeo funcionamiento												
TFG 3.x.5	Configuración de alertas y rutinas												
TFG 3.5.1	Comprobación gasto items no continables												

Tabla 3.3: Cronograma de tareas del TFG

Sabiendo ya el cronograma se pueden marcar la fecha de finalización de los distintos hitos.

Hitos		Fecha finalización
TFG 1.1.1	Análisis de requisitos no funcionales	12 de febrero de 2026
TFG 1.1.2	Análisis de requisitos funcionales	13 de febrero de 2026
TFG 1.2	Lista Items continuables	14 de febrero de 2026
TFG 2.1	Planificación	5 de marzo de 2026
TFG 2.2	Seguimiento y control	15 de junio de 2026
TFG 2.3	Tecnologías y herramientas	7 de marzo de 2026
TFG 2.4	Memoria	22 de junio de 2026
TFG 2.5	Defensa	3 de julio de 2026
TFG 3.1.1	Detección de items continuables	7 de marzo de 2026
TFG 3.1.2	Creación de contrato completo	28 de febrero de 2026
TFG 3.1.3	Configuración Credenciales (1)	5 de marzo de 2026
TFG 3.1.4	Testeo funcionamiento (1)	12 de marzo de 2026
TFG 3.1.5	Configuración de alertas y rutinas (1)	11 de marzo de 2026
TFG 3.2.1	Comprobación de /usage items continuable	25 de marzo de 2026
TFG 3.2.2	Comprobación de que contrato completo (2)	14 de marzo de 2026
TFG 3.2.3	Configuración Credenciales(2)	20 de marzo de 2026
TFG 3.3.4	Testeo funcionamiento (2)	29 de marzo de 2025
TFG 3.4.5	Configuración de alertas y rutinas (2)	27 de marzo de 2026
TFG 3.3.1	Comprobación gasto items continables	3 de abril de 2026
TFG 3.3.2	Comprobación de que contrato completo (3)	29 de marzo de 2026
TFG 3.3.3	Configuración Credenciales (3)	1 de abril de 2026
TFG 3.3.4	Testeo funcionamiento (3)	5 de abril de 2026
TFG 3.3.5	Configuración de alertas y rutinas (3)	6 de abril de 2026
TFG 3.4.1	Comprobación de /usage items no continuables	23 de abril de 2026
TFG 3.4.2	Creación de gasto de tockens automatico	20 de abril de 2026
TFG 3.4.3	Comprobación de que contrato completo (4)	16 de abril de 2026
TFG 3.4.4	Configuración Credenciales (4)	17 de abril de 2026
TFG 3.4.5	Testeo funcionamiento (4)	24 de abril de 2026
TFG 3.4.6	Configuración de alertas y rutinas (4)	25 de abril de 2026
TFG 3.5.1	Comprobación gasto items no continables	7 de mayo de 2026
TFG 3.5.2	Comprobación de que contrato completo (5)	23 de abril de 2026
TFG 3.5.3	Configuración Credenciales (5)	24 de abril de 2026
TFG 3.5.4	Testeo funcionamiento (5)	9 de mayo de 2026
TFG 3.5.5	Configuración de alertas y rutinas (5)	11 de mayo de 2026

Tabla 3.4: Hitos TFG

3.4. Plan de contingencias y riesgos

Siendo un proyecto de cierto tamaño, hay muchos factores que se pueden no tener en cuenta, debido a que cuanto más grande sea un proyecto más posibles problemas pueden ocurrir, por lo que para poder minimizar esos riesgos, se crea un plan de contingencia ante posibles riesgos, para proteger los activos más valiosos. Para hacerlo, se basará en el análisis de riesgos estudiado en la asignatura de Seguridad de esta titulación, impartida en esta universidad.

Plan de contingencias y riesgos					
Origen	**Riesgo**	**Probabilidad**	**Impacto**	**Solución**	**Tipo de medida**
Hardware	Destrucción o pérdida del ordenador de trabajo	baja	alto	Para evitar la pérdida de toda la información se hará una estrategia 2+1: dos copias en la nube con guardado de versiones (GitHub y GitLab) y una copia de seguridad periódica en un disco físico guardado y desconectado.	Preventiva
Humano	Siendo una sola persona la que participa en el proyecto, es posible que se encuentre indispuesto por diversas causas como enfermedad, por lo que se paralizaría el proyecto.	Media	Medio	Dejar espacio suficiente entre la finalización teórica del proyecto y la entrega real, para poder compensar posibles ausencias.	Preventiva

Tabla 3.5: Plan de contingencias y riesgos

Plan de contingencias y riesgos					
Origen	**Riesgo**	**Probabilidad**	**Impacto**	**Solución**	**Tipo de medida**
Datos	Trabajando con recursos valiosos puede haber intentos de robo de información, como contraseñas o tokens de APIs privadas de IONOS Cloud.	Media	Alto	Almacenamiento de estas credenciales en entornos seguros como KeePassXC y no incluirlas en el código.	Preventiva
Software	Cambios en las APIs usadas	Bajo	Alto	Usar versiones estables de las APIs y controlar posibles cambios para mantener el uso de las APIs antiguas o, en caso de estar obsoletas, cambiar a las nuevas.	Preventiva

Tabla 3.6: Plan de contingencias y riesgos

Capítulo 4

Iteraciones

4.1. Primera Iteración

4.1.1. Planificación

Siendo esta la primera iteración, es en la que más se desarrolla para que en las siguientes iteraciones todo resulte más sencillo. Esta iteración se basa en 4 puntos: La creación de la función para detectar los *items* continuables; las configuraciones iniciales en CouchDB; la creación de las primeras alarmas — después de crearlas, solo habrá que modificar su contenido en el resto de iteraciones — y la creación del contrato.

Uno de los objetivos fundamentales de este proyecto es la creación de la base del

proyecto y establecer las tecnologías fundamentales para iniciarlo. Con esto se engloba tanto el lenguaje que se usará como la decisión del método de persistencia del código. En este caso ambas decisiones son heredadas, se usa *JavaScript* porque es donde están desarrolladas las librerías internas del equipo de Billing de IONOS Cloud y para persistir se guardará en Gitlab de manera habitual y además por seguridad se guardará periodicamente tanto en github como en un disco duro físico.

Referido a la detección de los *items* continuables, se realiza en dos fases: la primera consiste en detectar solamente un SKU concreto y, cuando funcione, se extiende al resto de SKUs.

Tanto la configuración inicial de CouchDB, como la creación de las alarmas y la formalización del contrato se realizan con cuidadoso detalle, porque si se hacen bien en esta fase, en el resto de fases no hay que modificarlos mucho.

Verificación de SKUs con PostMan

Para la verificación de los SKUs disponibles en la API de IONOS Cloud, hemos optado por utilizar **PostManPostMan-Learning** como herramienta de prueba y automatización. Antes de comenzar a utilizarlo, es importante comprender su funcionamiento y las capacidades que ofrece para este proyecto.

PostMan es una plataforma de colaboración para el desarrollo de APIs que permite diseñar, probar, documentar y monitorizar APIs de manera eficiente. Para este trabajo, aprovecharemos especialmente su capacidad para ejecutar scripts de prueba en JavaScript y su sistema de colecciones que permite automatizar peticiones.

Conceptos básicos de PostMan para la verificación de SKUs

Antes de comenzar a usarlo, es importante entender algunos conceptos clave de PostMan:

- **Collections:** Son grupos de peticiones relacionadas que pueden organizarse en carpetas. En nuestro caso, crearemos una colección específica para las pruebas de verificación de SKUs de IONOS Cloud.
- **Environments y variables:** PostMan permite definir entornos con variables reutilizables. Utilizaremos variables de entorno para almacenar:
  - La URL base de la API de IONOS Cloud.
  - Los tokens de autenticación de forma segura.

  - Los resultados de las verificaciones para su uso en otras peticiones.

- **Pre-request scripts:** Scripts que se ejecutan *antes* de enviar una petición. Nos permitirán preparar datos, generar tokens temporales o configurar cabeceras de autenticación automáticamente.

- **Tests scripts:** Scripts que se ejecutan *después* de recibir la respuesta de la API. Aquí implementaremos la lógica de verificación de SKUs, comparando los recursos obtenidos con la lista predefinida de recursos requeridos.

- **Console:** Herramienta de depuración que muestra los logs generados por nuestros scripts. Utilizaremos `console.log()` para generar reportes detallados del estado de cada verificación.

- **Runner:** Funcionalidad que permite ejecutar colecciones completas de manera automática, ideal para realizar verificaciones periódicas sin intervención manual.

Base de datos con CouchDB

Para las configuraciones, exceptuando los tokens y las claves, hemos optado por utilizar **CouchDBCouchDB-Info**, una base de datos NoSQL orientada a documentos que destaca por su facilidad de integración con aplicaciones web.

Características principales de CouchDB

- **Almacenamiento basado en JSON:** Los datos se guardan como documentos JSON, lo que facilita su manipulación desde aplicaciones JavaScript.

- **API RESTful nativa:** CouchDB expone una API HTTP que permite realizar operaciones CRUD mediante peticiones estándar (GET, POST, PUT, DELETE), simplificando la comunicación con el backend.

- **MapReduce para consultas:** Implementa vistas mediante funciones MapReduce, permitiendo crear índices y consultas personalizadas sobre los documentos almacenados.

- **Replicación maestro-maestro:** Soporta sincronización bidireccional entre múltiples instancias, lo que facilita la distribución de datos y la tolerancia a fallos.

- **MVCC (Multiversion Concurrency Control):** Utiliza control de concurrencia mediante versiones, evitando bloqueos en lecturas y escrituras simultáneas.

4.1.2. Desarrollo y despliegue

Creación del contrato

Para crear el contrato se hace de manera manual, debido a que tras una breve investigación se concluyó que no había ningún método sencillo para crearlo de forma

automática, por lo que se consideró hacerlo de forma manual ya que el coste de tiempo iba a ser menor y el proceso más sencillo. El método elegido fue usar de manera combinada el DCD de IONOS con comprobaciones periódicas en PostMan. Para crear el contrato hay que tener primero una cuenta de IONOS Cloud y darse de alta en el DCD; debido a que este proceso se considera sencillo, no se desarrolla más. Si se quiere obtener más información, se puede consultar el Anexo 2.

El primer problema encontrado fue lograr vincular los recursos a sus SKUs, ya que el nombre de algunos SKUs no es intuitivo; para ello se empezó a usar la API interna de IONOS Cloud **billing-IONOSClaud**. Además, para usarla se necesitaba, aparte de la documentación, un token generado por el contrato —la forma de obtenerlo es sencilla y no es el objeto del proyecto; para más información se puede consultar el Anexo 2—. Una vez se sabe cómo obtener dicho token, hay que crear un plan de uso y control de dichos tokens, debido a que son información delicada.

Plan de gestión de tokens de autenticación

Para garantizar la seguridad en el acceso a la API de IONOS Cloud, se establece un plan de gestión de tokens de autenticación utilizando **KeePassXC** como gestor de contraseñas especializado.

Justificación de la herramienta

KeePassXC es un gestor de contraseñas de código abierto, auditado de forma independiente, que proporciona cifrado robusto (AES-256, Twofish o ChaCha20) para la información almacenada **keepassxc-audit**. Entre sus características más relevantes para este proyecto destacan:

- Capacidad de generar contraseñas y tokens seguros.
- Almacenamiento cifrado de secretos con protección en memoria.
- Generación integrada de TOTP (Time-based One-Time Passwords) **keepassxc-features**.
- Compatibilidad con archivos clave y YubiKey para autenticación de doble factor.
- Integración con navegadores mediante extensiones oficiales.

Ciclo de vida de los tokens en IONOS Cloud

Los tokens de autenticación en IONOS Cloud presentan las siguientes características **ionos-token-management**:

- Se generan a través del *Authentication Token Manager* en el DCD.
- Cada token tiene un TTL (*Time To Live*) configurable entre 1 hora y 365 días.

- El valor del token solo se muestra una única vez durante su generación.
- Es posible descargar el token como archivo de texto en el momento de creación.
- Se pueden generar hasta 100 tokens por cuenta.
- La eliminación de un token lo invalida inmediatamente, incluso antes de su fecha de expiración.

Procedimiento de gestión segura

1. **Generación del token:**
   - Acceder al DCD: Menú → Management → Token Manager.
   - Seleccionar TTL adecuado al caso de uso (para desarrollo: 30-90 días; para producción: evaluar periodicidad de rotación).
   - Generar el token y, en la pantalla de visualización, seleccionar *Download* para obtener el archivo de texto.
2. **Almacenamiento en KeePassXC:**
   - Crear una entrada específica para cada token en la base de datos de KeePassXC.
   - Campos a rellenar:
     - **Título:** "Token IONOS API - [Propósito]" (ej: "Token IONOS API - Desarrollo Iteración 1").
     - **Nombre de usuario:** Identificador del token (Token ID).
     - **Contraseña:** Pegar el valor del token.
     - **URL:** https://api.ionos.com (para integración con navegador).
     - **Notas:** Fecha de generación, TTL seleccionado, fecha de expiración calculada, propósito específico.
   - Adjuntar el archivo de texto descargado como archivo adjunto en la entrada.
   - Configurar la entrada para generar TOTP si se requiere autenticación adicional.
3. **Rotación y expiración:**
   - Configurar un recordatorio en el calendario para 7 días antes de la fecha de expiración.
   - En la fecha de rotación:
     a) Generar nuevo token en el DCD.
     b) Almacenarlo en KeePassXC siguiendo el procedimiento anterior.
     c) Eliminar el token antiguo desde el DCD.

d) Marcar la entrada antigua como expirada o eliminarla de la base de datos.

4. **Seguridad adicional:**

   - La base de datos de KeePassXC debe protegerse con una contraseña maestra robusta (mínimo 15 caracteres con alta entropía).
   - Habilitar el bloqueo automático de la base de datos tras periodo de inactividad.

Uso del Token

Teniendo ya clara la gestión de los tokens, se puede usar para la API. En concreto, en este contexto *"https://api.ionos.com/billing/contract/products"***billing-IONOSClaud**, sirviendo esta llamada para obtener la información de los productos que hay en el contrato. De este modo se puede comprobar cada elemento que se quiera encontrar.

Para el uso de llamadas a APIs se usa PostMan. Como primer SKU a detectar se opta por A01000 (IP estática). Para ello se añade una IP estática de forma manual en el DCD y, tras pasar el periodo que tarda en actualizarse la base de datos de la API, se hace una llamada y debe aparecer. En este caso debe salir algo similar a:

```
{
  "products": [
    {
      "meterId": "A01000",
      "meterDesc": "30d per static IP address",
      "deprecated": false,
      "unit": "30Days",
      "type": "consumption",
      "unitCost": {
        "quantity": "5",
        "unit": "EUR"
      }
    }
  ]
}
```

Listing 4.1: Ejemplo de producto en API de IONOS

Este proceso hecho de forma manual junto a añadir de manera manual los productos puede ser una tarea proclive a errores, debido a ser una tarea repetitiva, por lo que se opta por una solución sencilla y útil para el futuro: debido a que PostMan permite el uso de scripts, se puede crear uno pequeño que compruebe si están los SKUs deseados.

Creación del script de verificación

Se creó un script en JavaScript para PostMan que verifica los SKUs disponibles.

Funcionamiento

1. Define un objeto `requiredResources` con 14 categorías de SKUs (IPs estáticas, VMs, almacenamiento, bases de datos, etc.).

2. La función `checkResources()` compara cada SKU requerido con los devueltos por la API.

```
Object.keys(requiredResources).forEach(category => {
  const resources = requiredResources[category];
  results.available[category] = [];
  results.missing[category] = [];

  resources.forEach(resource => {
    const found = products.find(p => p.meterId ===
 resource);
    if (found) { results.available[category].push(resource);
    } else { results.missing[category].push(resource); }
  }); });
```

3. Clasifica los resultados en `available` (encontrados) y `missing` (no encontrados).

```
"🔄 Iniciando verificación de recursos..."
"Contrato: undefined"
GET https://api.ionos.com/billing/36348704/products
"✅ Response recibida - Productos: 215"
"===  REPORTE COMPLETO ==="
"✅ Static_IP: A01000"
"✅ VMs_Cores: C02000, C03000, C04000, C05000, C06000, CV1000"
"✅ VMs_RAM: R01000, RV1000"
"✅ VMs_Storage: S01000, S02000, S05000"
"✅ VMs_Snapshots: S03000"
"✅ Managed_K8s: MKC1000, MKCV1000, MKR1000, MKRV1000, MKS1000, MKS2000"
"✅ SQL_Licenses: CWSQL1001, CWSQL3001"
"✅ RedHat_Licenses: RHEL1200, RHEL2100, RHEL2200, RHEL2300"
"✅ DB_InMemory: DBIMC1000, DBIMR1000, DBIMS3000"
"✅ DB_MariaDB: DBMAB1000, DBMAC1000, DBMAR1000, DBMAS2000, DBMAS3000"
"✅ DB_MongoDB: DBMB1000, DBMB1100, DBMB1200, DBMB1300, DBMB1400, DBMB1500, DBMB1600, DBMB1610, DBMEC1000, DBMER1000, DBMES1000, DBMES2000, DBMES3000"
"✅ DB_PostgreSQL: DBPGC1000, DBPGR1000, DBPGS1000, DBPGS2000, DBPGS3000"
"✅ S3_Storage: S3SU1000, S3SU2000"
"✅ Backup: BA1100, BA2000, BA3000, BA4000, BU1300"
"X RedHat_Licenses FALTANTES: RHEL100"
```

Figura 4.1: Prueba Test

4. Detecta patrones especiales: VMs CUBE (contienen ÇUBES") y licencias Windows (empiezan por "WL").

5. Almacena los resultados en variables de entorno de PostMan.

6. Genera un reporte en consola con el estado de cada categoría.

7. Calcula estadísticas globales: total disponibles vs total requeridos.

```
"===  RESUMEN ==="
"Disponibles: 58/59"
"Tasa: 98%"
```

Figura 4.2: Prueba Resumen Test

8. Ejecuta tests automáticos (código 200, productos no vacíos).

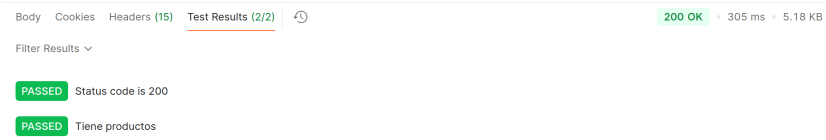


Figura 4.3: Prueba Cods. Test

Ventajas

Automatiza la verificación, proporciona trazabilidad, se integra en PostMan y es fácilmente mantenible. Además esta estructura es migrable al proyecto, permitiendo tener una parte hecha ya.

4.1.3. Integración del código

Configuración de Entornos

Al ser la primera iteración se deben configurar los entornos que se van a usar. Esto consiste en la creación del proyecto, su vinculación a Gitlab y Github, el plan de copias de seguridad en físico y la configuración inicial en CouchBD.

Despliegue del proyecto y vincluación a Gitlab

El proyecto se crea directamente desde gitlab debido a que se quiere integrar en un grupo de trabajo, en concreto en el equipo de *Cloud Billing* y *ERP* de IONOS Cloud, para que de ese modo, tras la finalizaciób del proyecto, todo el equipo pueda seguir con su mantenimiento. Debido a esto, se sigue además la estructura de proyectos propia de este equipo. La unica peculiaridad peculiaridad fue el uso de la VPN debido a que el Gitlab que se usa es el del equipo, siendo estos Alemanes.

Estrategia de copias de seguridad del código fuente usando GitHub y disco físico

Para garantizar la integridad y disponibilidad del código fuente desarrollado en este proyecto, se ha establecido una estrategia de copias de seguridad multicapa que complementa el sistema de control de versiones principal.

Uso de GitLab como sistema de control de versiones principal

GitLab actúa como el repositorio principal donde se gestiona el desarrollo activo del proyecto. Sus características incluyen:

- Control de versiones con historial completo de cambios.
- Gestión de ramas para el desarrollo de funcionalidades.
- Integración continua para pruebas automáticas.
- Documentación integrada mediante wikis.

GitHub como repositorio espejo de respaldo

A pesar de utilizar GitLab como plataforma principal, se ha configurado GitHub como repositorio secundario para proporcionar una capa adicional de redundancia. Esta decisión se fundamenta en:

- **Independencia entre plataformas:** Al estar alojados en servidores de diferentes proveedores, una caída o problema en GitLab no compromete el acceso al código.
- **Redundancia geográfica:** Los datos se replican en centros de datos de diferentes ubicaciones, protegiendo frente a desastres regionales. Previniendo uno de los posibles riesgos estudiado en en plan de riesgos.

Copia de seguridad en disco físico local

Complementando las copias en la nube, se realiza una copia de seguridad periódica en un disco físico externo siguiendo estos principios:

- **Frecuencia de respaldo:** Copia completa del repositorio cada 15 días y copia incremental de los cambios diarios.
- **Almacenamiento seguro:** El disco se mantiene en una ubicación física diferente a la del equipo de desarrollo, protegiendo frente a robo, incendio o fallo del equipo principal.
- **Verificación de integridad:** Tras cada backup, se ejecuta una comprobación automática mediante `git fsck` para garantizar que el repositorio no esté corrupto.

Configuración en CouchDB

Debido a que se está integrado en el grupo de trabajo de *Cloud Billing* y *ERP*, se usa su grupo de bases de datos. Lo único que hay que tener en cuenta es el uso de las credenciales, debido a que las credenciales para entrar a CouchBD son delicadas — ya que se accede a todas las configuraciones del equipo — se usa un protocolo similar al descrito en la sección Plan de gestión de tokens de autenticación.

Una vez creada la base de datos propia, se crea un documento para vincularlo con el proyecto.

Estructura del documento de configuración

El documento almacenado en CouchDB contiene la lógica y parámetros necesarios para las verificaciones:

- **Identificador:** `billing-reference-verifier` - Documento raíz que centraliza toda la configuración del verificador.
- **Programación de tareas (scheduler):** Define ejecuciones automáticas periódicas, sobre esto se desarrollará en mayor profundidad cuando se hable de ello en concreto.

Configuración específica para verificación de SKUs

El módulo `utilization` contiene la lista de SKUs a monitorizar, siguiendo la siguiente estructura:

```
{
  "skus_monitorizados": [
    "A01000", "C02000", "C03000", "C04000", "C05000", "C06000",
    [...],
    "CWSQL1001", "RHEL2100"
  ]
}
```

Listing 4.2: Almacenamiento SKUs en CouchDB

Estructura general del proyecto billing-reference-verifier

El proyecto `billing-reference-verifier` es una API REST interna desarrollada para IONOS cuyo propósito principal es verificar la utilización de recursos y validar facturas asociadas a contratos de facturación. Por lo que antes de centrarnos en conceptos tecnicos, se explicará la estructura geenral actual del proyecto. Centrandose en estructura y tecnologias.

Información general

- **Nombre:** billing-reference-verifier
- **Tecnología:** Node.js (versión $\geq$ 18.0.0) con framework interno `@internal/core`
- **Tipo:** API REST interna

Arquitectura de directorios

La estructura del proyecto se organiza de la siguiente manera:

```
billing-reference-verifier/
|-- package.json                  # Dependencias y scripts de Node.js
|-- README.md                     # Documentacion principal
|-- run.sh / start.sh             # Scripts de inicio del servicio
|-- config/
|   +-- nginx.conf                # Configuracion de proxy reverso
|-- docs/
|   |-- README.md                  # Documentacion adicional
|   +-- SUMMARY.md                 # Resumen de documentacion
|-- src/
|   |-- run.js                     # Punto de entrada (bootstrap)
|   |-- helpers/                    # Clases auxiliares y utilidades
|   |   |-- actions.js              # Constantes de eventos/acciones
|   |   |-- invoice.js              # Helper para operaciones de facturas
|   |   |-- schema.js               # Esquemas de validacion Joi
|   |   +-- utilization.js          # Helper para datos de utilizacion
|   +-- modules/                    # Modulos funcionales de la aplicacion
|       |-- utilization/
|           +-- index.js            # Logica de verificacion de utilizacion
+-- test/
    +-- test-usage-validate.js      # Pruebas automatizadas
```

Componentes principales

Punto de entrada (run.js)

Realiza el bootstrap de la aplicación utilizando el framework `@internal/core`, carga la configuración definida en `package.json` e inicializa el servidor y los módulos.

Módulos funcionales

- **utilization:** Verifica las cantidades diarias de uso frente a los valores esperados para los SKUs configurados.

Helpers

- **actions.js:** Define constantes para los eventos del mediador (como `APPLICATION_RUN`, `UTILIZATION_VERIFIER_COMPLETE`, etc.).
- **schema.js:** Contiene esquemas Joi para validar las respuestas de la API (`invoiceSchema`, `utilizationSchema`, `usageSchema`).
- **utilization.js:** Implementa la clase `UtilizationHelper` para obtener datos de los endpoints `/utilization` y `/usage` de la API de facturación.

Endpoints REST disponibles

- `GET /data/v1/utilization/check/:contractId` — Verifica la presencia de ítems configurados.
- `GET /data/v1/utilization/debug` — Información de diagnóstico.
- `POST /data/v1/invoice/manual-test/:contractId` — Test manual de validación.

Flujo de datos

1. Una petición HTTP llega al módulo `data-api`, que valida los headers (incluyendo el token Bearer).
2. El mediador emite un evento al módulo correspondiente según el endpoint solicitado.
3. El módulo utiliza los helpers para realizar llamadas a la `billing-api` de IONOS.
4. Las respuestas se validan mediante los esquemas Joi definidos en `schema.js`.
5. Se devuelve una respuesta JSON estructurada con el resultado, resumen y timestamp.

Dependencias externas

- **@internal/core:** Framework base que proporciona servidor Koa, cliente REST y utilidades.
- **helmet:** Middleware de seguridad para cabeceras HTTP.
- **@internal/dev:** Herramientas de desarrollo (eslint, nodemon, etc.).
- **joi:** Validación de esquemas (integrado a través de @internal/core).

Patrones de diseño utilizados

- **Módulos desacoplados:** Cada módulo hereda de una clase `Module` base.
- **Mediator Pattern:** Comunicación entre módulos mediante eventos.
- **Factory Pattern:** Uso de `createHelper()` para instanciar helpers.
- **Dependency Injection:** Container para servicios.
- **Schema Validation:** Joi para validar estructuras de datos.

Autenticación

Todas las peticiones requieren el header `Authorization: Bearer <token>`. Este token se transmite a la `billing-api` de IONOS para autenticar las operaciones y se gestiona como ya se explicá en el plan de gestión de tokens de autenticación.

4.1.4. Implementación de función de Detección de items continuables

El desarollo de esta función se explicará por partes

Anexo 1

Lista de SKUs

Lista de elementos continuables no obsoletos, Este trabajo usa como base de la información de esta lista documentación propia de IONOS Claud **billing-IONOSClaud**.

- **VDC VMs**
  - **Static IP**
    - **A01000:** IP estática, una dirección numérica única, fija y permanente asignada a un dispositivo en una red que no cambia con el tiempo.
  - **VMs/cores**
    - **CO2000:** *"1h core-pair Intel Xeon Gen 5 (Haswell)"*
    - **C03000:** *"1h core-pair Intel Xeon Gen 5 (Skylake)"*
    - **C04000:** *"1h core-pair Intel Xeon Gen 5 (Ice Lake)"*
    - **C05000:** *"1h core-pair AMD EPYC Gen 3 (Milan)"*
    - **C06000:** *"1h core Intel Xeon Gen 6 (Sierra Forest)"*
    - **CV1000:** *"1h vCPU Processing Unit"*
  - **VMs/RAM:**
    - **R01000:** *"1h per GB RAM"*
    - **RV1000:** *"1h per GB RAM vCPU"*
  - **VMs/Storage**
    - **S01000:** *"30d per 1GB HDD Storage"*
    - **S02000:** *"30d per 1GB SSD Premium Storage"*
    - **S05000:** *"30d per 1GB SSD Standard Storage"*
  - **VMs/Snapshots**
    - **S03000:** *"30d per 1GB HDD Snapshot"*
  - **CUBES2**
    - **CUBES2200:** *"1h of IONOS Cloud Basic Cube S with configuration of vCPU: 2, RAM: 4096 MB, Storage: 122880 MB"*
  - **mk8s (managed kubernetes)**
    - **MKC1000:** *"1h Dedicated Core Processing Unit for Managed Kubernetes"*
    - **MKCV1000:** *"1h vCPU Processing Unit for Managed Kubernetes"*
    - **MKR1000:** *"1h per GB RAM for Managed Kubernetes (Dedicated Core)"*

- **MKRV1000:** *"1h per GB RAM for Managed Kubernetes (vCPU)"*
- **MKS1000:** *"30d per 1GB HDD Storage for Managed Kubernetes"*
- **MKS2000:** *"30d per 1GB SSD Premium Storage for Managed Kubernetes"*

- **Licenses**
  - **SQL licenses :**
    - **CWSQL1001 :** *"1h per 2 cores License MS SQL Standard"*
    - **CWSQL3001 :** *"1h per 2 cores License MS SQL Web"*
  - **RedHat licenses :**
    - **RHEL2100 :** *"1h per vCPU/ Core Red Hat Enterprise Linux Server Small Virtual Node"*
    - **RHEL2200 :** *"1h per vCPU/ Core Red Hat Enterprise Linux Server Medium Virtual Node"*
- **Managed DBs**
  - **Managed DBs (in-memory) :**
    - **DBIMC1000 :** *"1h core for In-Memory DB"*
    - **DBIMR1000 :** *"1h per GB RAM for In-Memory DB"*
    - **DBIMS3000 :** *"30d per GB SSD Premium Storage for In-Memory DB"*
  - **MariaDB**
    - **DBMAB1000 :** *"30d per GB Backup on Object Storage for MariaDB"*
    - **DBMAC1000 :** *"1h Core for MariaDB"*
    - **DBMAR1000 :** *"1h per GB RAM for MariaDB"*
    - **DBMAS3000 :** *"30d per GB SSD Premium Storage for MariaDB"*
  - **MongoDB**
    - **DBMB1000 :** *"1h of MongoDB Business XS Instance"*
    - **DBMB1100 :** *"1h of MongoDB Business S Instance"*
    - **DBMB1200 :** *"1h of MongoDB Business M Instance"*
    - **DBMB1300 :** *"1h of MongoDB Business L Instance"*
    - **DBMB1400 :** *"1h of MongoDB Business XL Instance*
    - **DBMB1500 :** *"1h of MongoDB Business XXL Instance*
    - **DBMB1600 :** *"1h of MongoDB Business 4XL Instance"*
    - **DBMB1610 :** *"1h of MongoDB Business 4XL-S Instance"*
    - **DBMEC1000 :** *"1h Core for MongoDB Enterprise"*
    - **DBMER1000 :** *"1h per GB RAM for MongoDB Enterprise"*
    - **DBMES1000 :** *"30d per GB HDD Storage for MongoDB Enterprise"*

    - **DBMES2000 :** *"30d per GB SSD Standard Storage for MongoDB Enterprise"*
    - **DBMES3000 :** *"30d per GB SSD Premium Storage for MongoDB Enterprise"*
  - **PostreSQL**
    - **DBPGC1000 :** *"1h core for PostgreSQL DBaaS"*
    - **DBPGR1000 :** *"1h per GB RAM for PostgreSQL DBaaS"*
    - **DBPGS1000 :** *"30d per GB HDD Storage for PostgreSQL DBaaS*
    - **DBPGS2000 :** *"30d per GB SSD Standard Storage for PostgreSQL DBaaS"*
    - **DBPGS3000 :** *"30d per GB SSD Premium Storage for PostgreSQL DBaaS"*

- **S3 Object Storage**
  - **S3 Cloudian :**
    - **S3SU1000 :** *"30d per 1GB Object Storage for user-owned buckets"*
  - **S3 Ceph :**
    - **S3SU2000 :** *"STANDARD storage class, 30d per 1GB Object Storage for contract-owned buckets"*

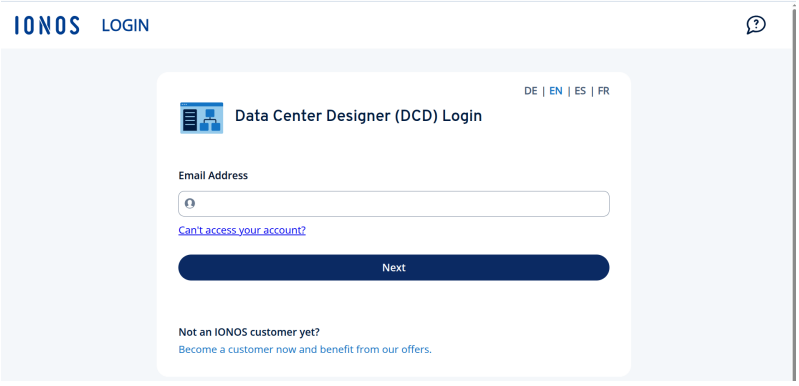
- **Container registry**
  - **Container registry storage:**
    - **CRSU1000 :** *"30d per 1GB Container Registry Storage"*
    - **CRVS1000 :** *"30days per 1 GB Container Registry Vulnerability Scanning"*

Anexo 2

Crear cuenta y obtener token

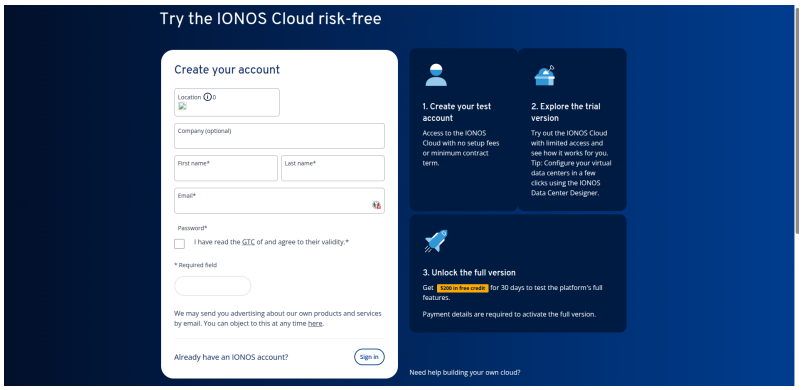
2.1. Creación de la Cuenta

Para la creación se puede hacer desde el inicio de sesión del DCD, que es el siguiente. Cuando se tenga cuenta solo hará falta añadir el correo en email address e iniciar sesión



Interfaz inicio sesión DCD IONOS Claud

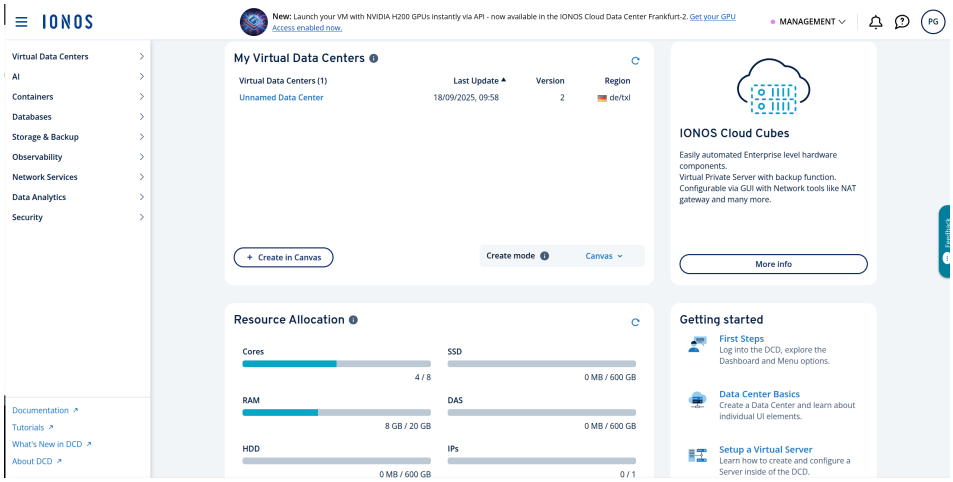
como es el estandar. En el caso actual se debe dirigir a "*Not an IONOS customer yet?*, dirgiendo este enlace a la web de creación de cuentas.



Crear cuenta IONOS Claud

Como se puede ver en la imagen, lo unico que hay que hacer es añadir datos estandar

y posteriormente aceptar sus políticas de privacidad. Una vez creada la cuenta, se vuelve a la parte anterior y se inicia sesión con el correo al que se ha asociado la cuenta. Una ve iniciado se entrará en una pagina similar a la mostrada actualmente.

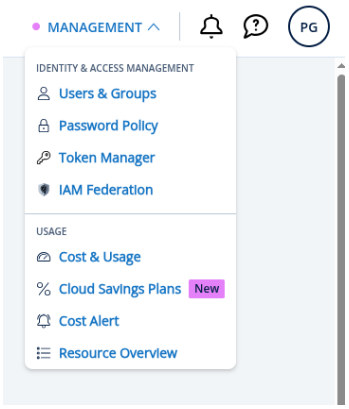


Iterfaz inicio DCD IONOS Claud

Una vez dentro ya se habra creado el contrato y se tendrá la cuenta, completando el primer paso.

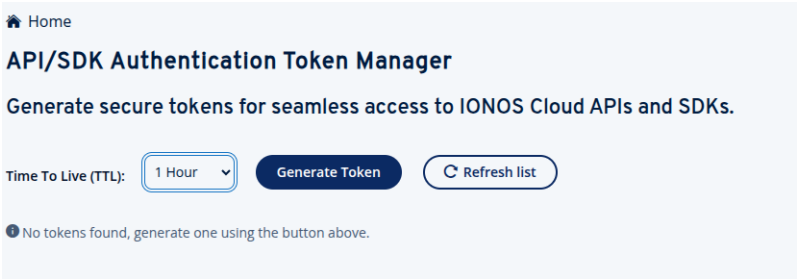
2.2. Obtener token

Una vez dentro de la interfaz, se debe pinchar en "*MANAGEMENT*"donde se desplegará un menú en el que se deberá seleccionar *"Token Manager"*. Una vez dentro,



Barra de gestión del DCD

es muy intuitivo como generar un token, se debe seleccionar la duración de validez del token, esta dependerá del uso que se quiera dar a dicho token. Una vez creado se debe almacenar correctamente dicho token, debido a que es información valiosa y se debe llevar acabo un protocolo te protección para este.



Interfaz de gestión de tokens